

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

AN AUTONOMOUS INSTITUTION

(Accredited by NAAC A+ Approved by AICTE New Delhi & Permanently Affiliated to JNTUH)

Aziz Nagar Gate, C.B. Post, Hyderabad-500 075

DEPARTMENT OF INFORMATION TECHNOLOGY

LAB PROJECT REPORT



Roll No	:	24911A1269
Name	:	Anthati Vaani
Project Title	:	Banking Management System
Course Name	:	Object Oriented Programming through Java Lab
Course Code	:	A224592
Academic Year	:	2025-26
Regulations	:	R22



VIDYA JYOTHI
INSTITUTE OF TECHNOLOGY
AN AUTONOMOUS INSTITUTION

CERTIFICATE

This is to certify that the Object Oriented Programming through Java Lab project report entitled “ **Banking Management System** ” submitted by **Anthati Vaani (24911A1269)** to Vidya Jyothi Institute of Technology (An Autonomous Institution), Hyderabad, of B. Tech. II year, II Semester in Information Technology a Bonafide record of Lab project work carried out by them under my supervision.

Signature of Faculty

Mr. B Eswar Babu

Associate Professor

Signature of Head of the Department

Dr. A. Obulesu

Professor

Abstract

The Banking Management System is a Java-based application developed to simplify and automate basic banking operations. In traditional banking systems, managing deposits, withdrawals, balance inquiries, and transaction records manually can be time-consuming and prone to errors. The need for a secure and efficient system that can handle multiple user requests simultaneously forms the basis of this project.

The main objective of this project is to provide a reliable banking solution that allows users to perform account operations such as depositing money, withdrawing funds, checking account balances, and viewing transaction history. The system also ensures data consistency and security during concurrent account operations through synchronization techniques.

The project is developed using Java as the programming language and utilizes JDBC (Java Database Connectivity) for interaction with a MySQL database. Core Java concepts such as multithreading, synchronization, exception handling, and object-oriented programming are implemented to enhance the functionality and reliability of the system.

The expected outcome of the project is a user-friendly banking application capable of efficiently managing account information and transaction records. It ensures accurate processing of banking operations, prevents data inconsistencies during simultaneous transactions, and provides secure access to account details. The system demonstrates the practical application of advanced Java concepts and serves as an effective learning tool for understanding database connectivity and concurrent programming in real-world banking environments.

Table Of Contents

Chapter No	Title	Page No
1	Introduction	1
2	System Analysis	6
3	System Design	8
4	Implementation	11
5	Testing	16
6	Results	18
7	Conclusion	22

Chapter 1 – Introduction

1.1 Project Overview

The Banking Management System is a Java-based application designed to automate basic banking operations such as depositing money, withdrawing funds, checking account balances, and maintaining transaction history. The system provides a simple and efficient way to manage customer account information and banking transactions.

In traditional banking systems, maintaining records manually can be time-consuming and may lead to errors. This project eliminates such issues by providing a computerized solution that performs banking operations accurately and efficiently. The system uses JDBC for database connectivity, allowing secure storage and retrieval of account information and transaction records.

The project also demonstrates the use of multithreading and synchronization to handle concurrent account operations safely. This ensures that multiple transactions can be processed without affecting data consistency.

Real-World Applications

- Banking and financial institutions
- Digital banking systems
- Online transaction management
- Customer account management systems
- Financial record maintenance

1.2 Objectives

The main objectives of the Banking Management System are:

1. To automate manual banking operations.
2. To reduce human errors in transaction processing.
3. To provide quick and accurate account management.
4. To maintain transaction history for future reference.
5. To ensure secure handling of account operations using synchronization.

6. To demonstrate the implementation of JDBC connectivity in Java applications.
7. To improve the efficiency and reliability of banking services.

1.3 Scope of the Project

The Banking Management System is intended for use by bank employees and customers to perform basic banking activities efficiently. The system allows users to manage account information, perform transactions, and view transaction history in a secure environment.

The project can be further enhanced in the future by adding advanced features such as:

- User authentication and authorization
- Online banking services
- Fund transfer between accounts
- ATM integration
- Mobile banking support
- Interest calculation and loan management
- Advanced reporting and analytics

The system is scalable and can be expanded to support multiple branches, larger customer databases, and additional banking functionalities as required.

Chapter 2 – System Analysis

2.1 Functional Requirements

The Banking Management System must provide the following functionalities:

User Account Management

- Create and manage bank accounts.
- Store customer account details securely.

Deposit Money

- Allow users to deposit money into their accounts.
- Update the account balance automatically.

Withdraw Money

- Allow users to withdraw money from their accounts.
- Check available balance before withdrawal.

Balance Enquiry

- Display the current account balance.
- Provide accurate and updated account information.

Transaction History

- Record all deposit and withdrawal transactions.
- Display transaction details when requested.

Database Connectivity

- Store account and transaction details in a MySQL database using JDBC.
- Retrieve information whenever required.

Concurrent Transaction Handling

- Support multiple transactions using multithreading.
- Ensure safe account operations through synchronization.

2.2 Non-Functional Requirements

Security

- Protect account information from unauthorized access.
- Ensure secure database connectivity.

Performance

- Process transactions quickly and efficiently.
- Minimize response time for banking operations.

Reliability

- Maintain accurate account balances.
- Prevent data loss during transactions.

Scalability

- Support additional banking features in the future.
- Handle an increasing number of users and transactions.

Maintainability

- Allow easy modification and enhancement of the system.
- Follow modular programming practices.

Availability

- Ensure the system is accessible whenever required.

2.3 Software Requirements

Requirement	Details
Operating System	Windows / Linux
Programming Language	Java
IDE	NetBeans / Eclipse / IntelliJ IDEA
Database	MySQL
JDBC Driver	MySQL Connector/J
JDK Version	JDK 8 or Above
Version Control	Git & GitHub

2.4 Hardware Requirements

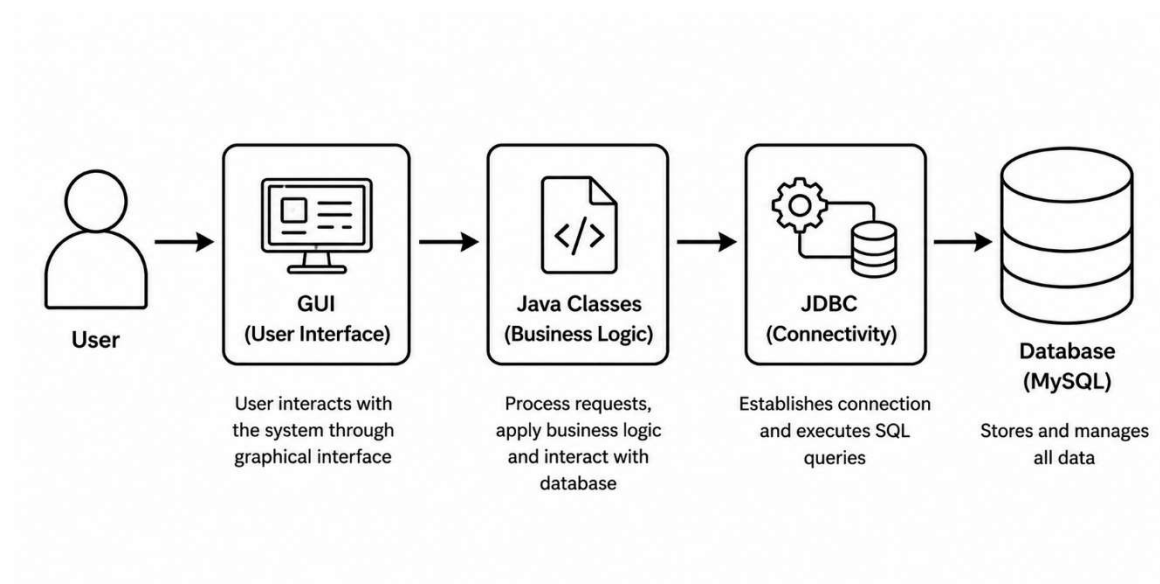
Requirement	Details
RAM	4 GB or Above
Processor	Intel i3 or Above
Storage	500 GB HDD / SSD
Monitor	Standard Display
Keyboard	Standard Keyboard
Mouse	Standard Mouse

Internet Connection Required for GitHub and Software Downloads

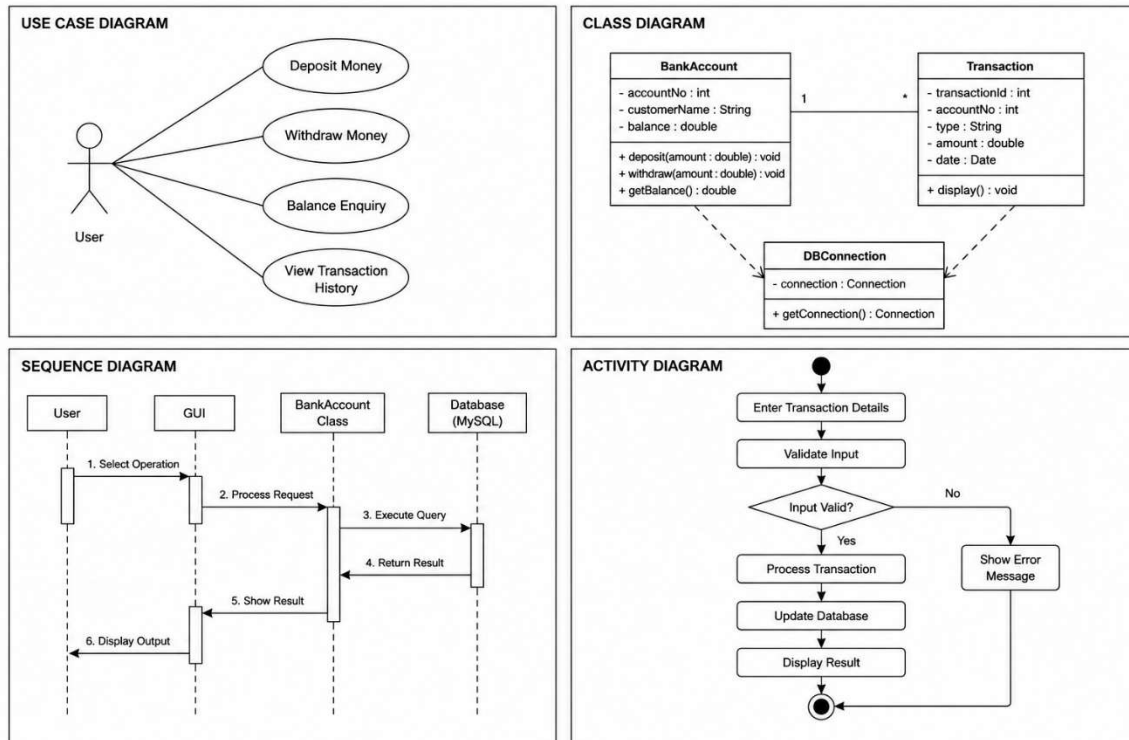
The above hardware and software requirements are sufficient for developing, testing, and running the Banking Management System efficiently.

Chapter 3 – System Design

3.1 Architecture Diagram



3.2 UML Diagrams



3.3 Database Design

The Banking Management System uses a MySQL database to store account information and transaction records. The database consists of two main tables.

Database Name: bankdb

Table 1: accounts

Table Name Purpose

accounts Stores customer account details and current account balance

Fields in accounts Table

Field Name	Data Type	Description
account_no	INT	Unique account number (Primary Key)
customer_name	VARCHAR(50)	Name of the customer
balance	DOUBLE	Current account balance

Table 2: transactions

Table Name Purpose

transactions Stores deposit and withdrawal transaction records

Fields in transactions Table

Field Name	Data Type	Description
transaction_id	INT	Unique transaction ID (Primary Key)
account_no	INT	Account number
transaction_type	VARCHAR(20)	Deposit or Withdraw
amount	DOUBLE	Transaction amount

transaction_date TIMESTAMP Date and time of transaction

Database Relationship

- One account can have multiple transactions.
- The account_no field in the transactions table refers to the account_no field in the accounts table.
- This relationship helps maintain transaction history for each customer account.

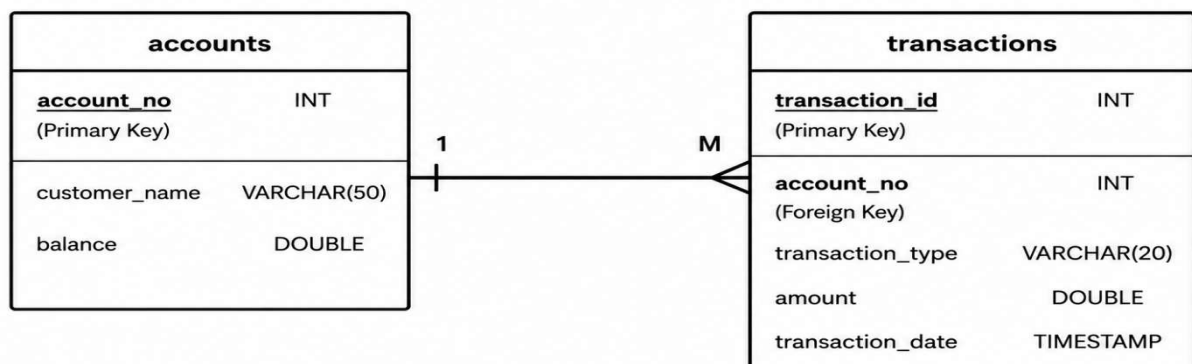
Database Tables Summary

Table Name Purpose

accounts Store customer account information and balance

transactions Store transaction history including deposits and withdrawals

The database design ensures efficient storage, retrieval, and management of banking data while maintaining accuracy and consistency.



One account can have multiple transactions.
The **account_no** in **transactions** table references **account_no** in **accounts** table.

Chapter 4 – Implementation

4.1 Modules Description

Module	Description
Account Management Module	Creates and manages customer accounts.
Deposit Module	Allows users to deposit money into their accounts.
Withdrawal Module	Allows users to withdraw money from their accounts after balance verification.
Balance Enquiry Module	Displays the current account balance of the customer.
Transaction History Module	Maintains and displays records of all deposits and withdrawals.
Database Module	Stores and retrieves account and transaction information using MySQL and JDBC.
Synchronization Module	Ensures safe account operations during concurrent transactions.
Exception Handling Module	Handles errors such as insufficient balance, invalid input, and database connection failures.
Report Module	Generates various reports related to accounts and transactions.

4.2 Important Java Concepts Used

1. Classes and Objects

Classes and objects are used to represent bank accounts, transactions, and database operations. They form the foundation of the Banking Management System.

2. Inheritance

Inheritance can be used to extend common banking functionalities and promote code reusability.

3. Polymorphism

Polymorphism allows methods to perform different tasks based on the object that invokes them, providing flexibility in program design.

4. Exception Handling

Exception handling is used to manage runtime errors such as insufficient balance, invalid account numbers, invalid transaction amounts, and database connection failures.

5. Multithreading

Multithreading enables multiple banking operations to be performed simultaneously, improving system efficiency and responsiveness.

6. Synchronization

Synchronization ensures that multiple threads do not access or modify the same account data at the same time, maintaining data consistency.

7. Collections

Collections can be used to store and manage transaction records efficiently within the application.

8. File Handling

File handling can be used to maintain logs and backup transaction records for future reference.

9. AWT/Swing

AWT and Swing are used to create the graphical user interface of the Banking Management System.

10. JDBC

JDBC (Java Database Connectivity) is used to connect the Java application with the MySQL database for storing and retrieving account information and transaction details.

4.3 Code Snippets

Database Connection Code (JDBC)

```
import java.sql.Connection;
```

```
import java.sql.DriverManager;
```

```
public class DBConnection {
```

```
    private static final String URL =
```

```
        "jdbc:mysql://localhost:3306/bankdb";
```

```

private static final String USER = "root";
private static final String PASSWORD = "root";

public static Connection getConnection() throws Exception {

    Class.forName("com.mysql.cj.jdbc.Driver");

    return DriverManager.getConnection(
        URL,
        USER,
        PASSWORD
    );
}
}

```

Deposit Method

```

public synchronized void deposit(double amount) {
    balance += amount;
    System.out.println( "Amount Deposited: " + amount );
}

```

Withdrawal Method

```

public synchronized void withdraw(double amount)
    throws Exception {

    if(balance < amount) {

        throw new Exception(
            "Insufficient Balance"

```

```

        );
    }

    balance -= amount;

    System.out.println(
        "Amount Withdrawn: " + amount
    );
}

```

Multithreading Example

```

class DepositThread extends Thread {

    BankAccount account;

    DepositThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        account.deposit(5000);
    }
}

```

Balance Enquiry

```

public double getBalance() {

    return balance;
}

```

Transaction History Display

```
public void displayTransaction(
```

```
    String type,
```

```
    double amount) {
```

```
    System.out.println(
```

```
        type + " : " + amount
```

```
    );
```

```
}
```

The implementation successfully integrates JDBC, multithreading, synchronization, exception handling, and object-oriented programming concepts to develop a reliable Banking Management System.

Chapter 5 – Testing

5.1 Test Cases

Test Case ID	Input	Expected Output	Result
TC01	Deposit Amount	Amount Deposited Successfully	Pass
TC02	Withdraw Valid Amount	Amount Withdrawn Successfully	Pass
TC03	Withdraw Amount Greater Than Balance	Insufficient Balance Error Message	Pass
TC04	Balance Enquiry	Display Current Balance	Pass
TC05	View Transaction History	Display Transaction Records	Pass
TC06	Database Connection	Database Connected Successfully	Pass
TC07	Invalid Input Amount	Error Message Displayed	Pass
TC08	Concurrent Transactions	Data Consistency Maintained	Pass

5.2 Error Handling

The Banking Management System uses exception handling to ensure smooth execution and prevent system failures. The following exceptions are handled:

- `NumberFormatException`
 - Occurs when the user enters invalid numeric values.
- `SQLException`
 - Occurs when there is an error while connecting to or accessing the MySQL database.
- `NullPointerException`
 - Occurs when an object reference is null.

- ArithmeticException
 - Occurs during invalid arithmetic operations.
- ClassNotFoundException
 - Occurs when the JDBC driver is not found.
- Exception
 - Used as a general exception handler for unexpected runtime errors.

Proper exception handling improves the reliability, stability, and user experience of the Banking Management System.

Chapter 6 – Results and Screenshots

Figure 6.1: Login Screen

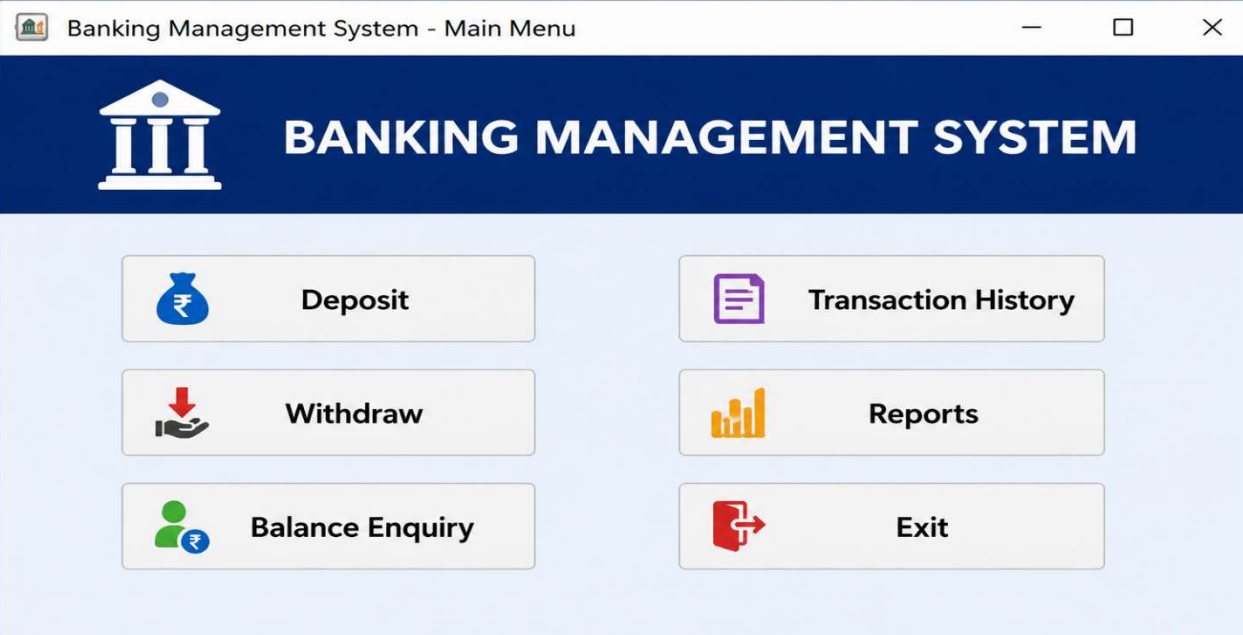


The screenshot shows a web browser window titled "Banking Management System - Login". The page has a light blue background. At the top center, the text "BANKING MANAGEMENT SYSTEM" is displayed in bold blue capital letters. Below this text is a blue icon of a classical building with four columns and a pediment, with the word "BANK" written on its roof. To the right of the icon are two input fields. The first is labeled "Username :" and contains the text "admin". The second is labeled "Password :" and contains ten black dots. Below the input fields are two buttons: a blue button with a right-pointing arrow and the text "Login", and a grey button with a circular arrow icon and the text "Reset". At the bottom of the page, there is a small shield icon followed by the text "Enter your credentials to access the system".

Description:

The login screen allows users to enter their credentials and access the Banking Management System securely.

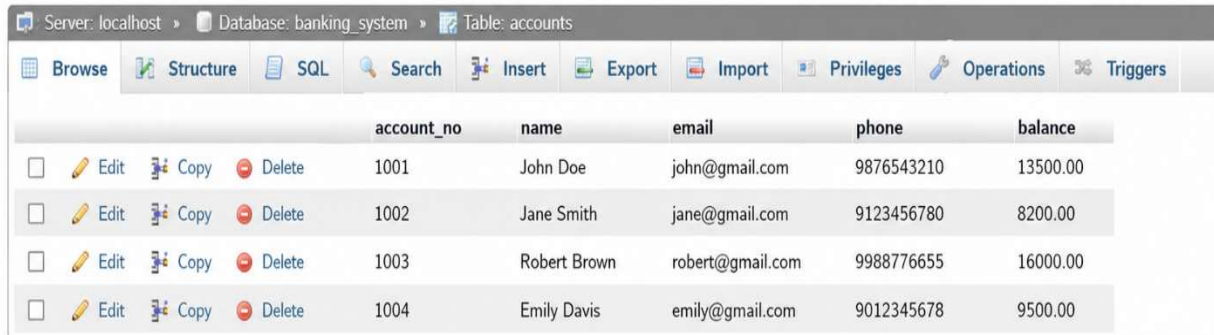
Figure 6.2: Main Menu Screen



The screenshot shows a web browser window titled "Banking Management System - Main Menu". The page has a dark blue header bar. On the left side of the header is a white icon of a classical building with four columns. To the right of the icon, the text "BANKING MANAGEMENT SYSTEM" is displayed in white capital letters. Below the header bar, the page has a light blue background. There are six buttons arranged in two columns. The left column contains three buttons: "Deposit" with a blue bag icon, "Withdraw" with a red arrow pointing down into a hand icon, and "Balance Enquiry" with a green person icon and a blue bag icon. The right column contains three buttons: "Transaction History" with a purple document icon, "Reports" with an orange bar chart icon, and "Exit" with a red door icon.

Description:

The main menu provides options such as Deposit, Withdraw, Balance Enquiry, Transaction History, and Exit.

Database Tables**Figure 6.3: Accounts Table**

The screenshot shows a database management interface with the following components:

- Navigation bar: Server: localhost > Database: banking_system > Table: accounts
- Toolbar: Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, Triggers
- Table structure: account_no, name, email, phone, balance
- Data rows: 4 records for accounts 1001 to 1004.

				account_no	name	email	phone	balance
☐	Edit	Copy	Delete	1001	John Doe	john@gmail.com	9876543210	13500.00
☐	Edit	Copy	Delete	1002	Jane Smith	jane@gmail.com	9123456780	8200.00
☐	Edit	Copy	Delete	1003	Robert Brown	robert@gmail.com	9988776655	16000.00
☐	Edit	Copy	Delete	1004	Emily Davis	emily@gmail.com	9012345678	9500.00

Figure 6.3: Transactions Table

Server: localhost > Database: banking_system > Table: transactions

Browse

Structure

SQL

Search

Insert

Export

Import

Privileges

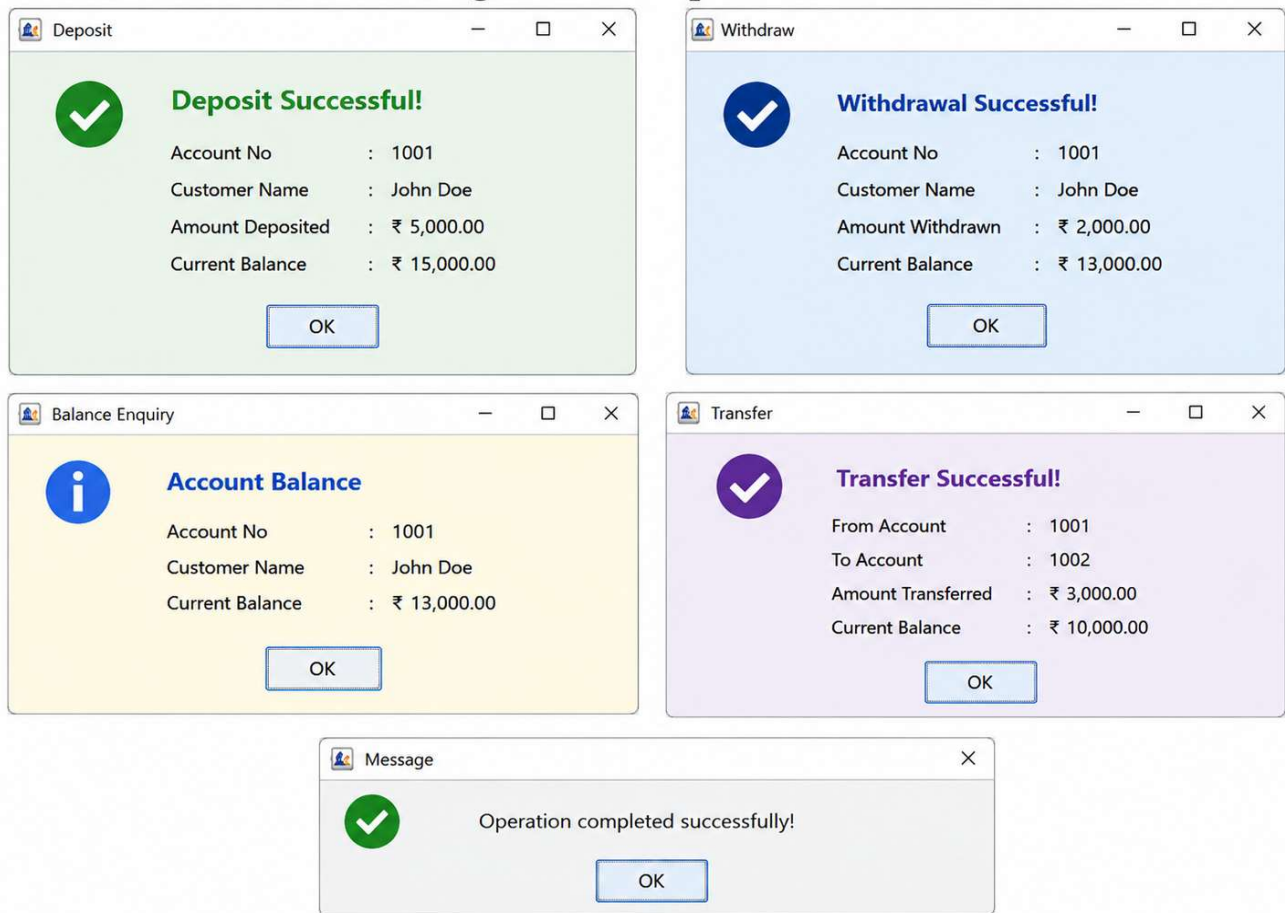
Operations

Triggers

					txn_id	account_no	type	amount	txn_date
☐	 Edit	 Copy	 Delete	1	1001	Deposit	5000.00	2025-05-18 10:15:30	
☐	 Edit	 Copy	 Delete	2	1001	Withdraw	2000.00	2025-05-18 11:05:20	
☐	 Edit	 Copy	 Delete	3	1001	Deposit	3000.00	2025-05-18 12:20:45	
☐	 Edit	 Copy	 Delete	4	1001	Withdraw	1000.00	2025-05-18 13:15:10	
☐	 Edit	 Copy	 Delete	5	1001	Deposit	2500.00	2025-05-18 14:30:05	
☐	 Edit	 Copy	 Delete	6	1002	Deposit	4000.00	2025-05-18 10:45:12	
☐	 Edit	 Copy	 Delete	7	1002	Withdraw	1500.00	2025-05-18 11:50:33	

Description: This screen displays the database tables used in the system, including account details and transaction records stored in MySQL.

Figure 6.4: Output Screens



Description: This screen shows the successful execution of banking operations such as deposit, withdrawal, and balance enquiry.

- Reports

Figure 6.5

Transaction Reports

BANKING MANAGEMENT SYSTEM

TRANSACTION REPORT

Report Filters

From Date : 01/05/2025

To Date : 31/05/2025

Account No : All Accounts

View Report

Txn ID	Account No	Customer Name	Type	Amount (₹)	Balance (₹)	Date & Time	Description
1	1001	John Doe	Deposit	5,000.00	15,000.00	18/05/2025 10:15:30	Cash Deposit
2	1001	John Doe	Withdraw	2,000.00	13,000.00	18/05/2025 11:05:20	ATM Withdrawal
3	1001	John Doe	Deposit	3,000.00	16,000.00	18/05/2025 12:20:45	Online Transfer
4	1001	John Doe	Withdraw	1,000.00	15,000.00	18/05/2025 13:15:10	Cash Withdrawal
5	1002	Jane Smith	Deposit	4,000.00	12,000.00	18/05/2025 14:30:05	Cash Deposit
6	1002	Jane Smith	Withdraw	1,500.00	10,500.00	18/05/2025 15:40:18	ATM Withdrawal
7	1003	Robert Brown	Deposit	2,500.00	18,500.00	18/05/2025 16:10:33	Cash Deposit
8	1003	Robert Brown	Withdraw	3,000.00	15,500.00	18/05/2025 17:25:50	Online Payment

Summary (All Accounts)

Total Deposits : ₹ 14,500.00

Total Withdrawals : ₹ 7,500.00

Net Amount : ₹ 7,000.00

Opening Balance : ₹ 0.00

Closing Balance : ₹ 23,500.00

Print Report

Export to Excel

Close

Description: This screen generates transaction reports and account summaries, helping users view and analyze banking records efficiently.

Chapter 7 – Conclusion and Future Scope

Conclusion

The Banking Management System was successfully designed and implemented using Java, JDBC, and MySQL. The system provides essential banking functionalities such as deposit, withdrawal, balance enquiry, and transaction history management. Synchronization was implemented to ensure safe account operations during concurrent transactions, while exception handling improved the reliability and robustness of the application.

This project helped in understanding important Java concepts including Object-Oriented Programming, Multithreading, Synchronization, Exception Handling, Collections, Swing GUI development, and JDBC database connectivity. The project successfully met its objectives by automating banking operations, reducing manual effort, improving accuracy, and ensuring efficient data management.

Future Scope

The functionality of the Banking Management System can be enhanced in the future through the following improvements:

- Cloud Integration for secure online data storage and access.
- Mobile Application Support for accessing banking services through smartphones.
- Online Payment Gateway Integration for digital payments and fund transfers.
- User Authentication and Role-Based Access Control for enhanced security.
- SMS and Email Notifications for transaction updates and account alerts.
- Internet Banking Features such as online fund transfer and bill payments.
- Advanced Reporting and Analytics for better financial insights.
- Integration with QR Code and UPI Payment Systems.

These enhancements would make the system more secure, scalable, and suitable for real-world banking applications.

References

- Herbert Schildt, *Java: The Complete Reference*, Oracle Press, Latest Edition.
- Bruce Eckel, *Thinking in Java*, Prentice Hall.
- MySQL Workbench Documentation, Oracle Corporation.
- Oracle Java Documentation, Oracle Corporation.
- JDBC (Java Database Connectivity) Documentation, Oracle Corporation.
- NetBeans IDE Documentation / Eclipse IDE Documentation.
- MySQL Database Documentation, Oracle Corporation.
- Java SE Development Kit (JDK) Documentation.